

Assignment 5: ADTs and Higher-Order Functions

CAS CS 320: Concepts of Programming Languages

due Thursday, October 8 by 8:00PM

For the OCaml problems, you'll put your solutions in the file `lib/assign5.ml`. See the file `test/test_assign5.ml` for example behavior of each function. You'll submit your solutions via Gradescope, where your code will be checked against a collection of hidden tests. You must pass **at least half** of the Gradescope unit tests on each problem for the assignment to be considered attempted. Your code must also follow the guidelines of the course for it to be considered attempted (i.e., it must use the course standard library, and it may not use any references, arrays, mutable fields, or for/while loops).

For the written problems, you'll submit your solutions via Gradescope as a single PDF file. They must be hand-written and exceptionally neat, and the final answer to each problem must be clear, e.g., surrounded by a box. You must cite your sources per the instructions in the course manual.

You're required to submit solutions to every problem in this assignment to receive full credit, but we will only provide detailed feedback on **bolded** problems. Since the assignment is graded on participation, we ask that you *try at least the bolded problems with as few external aids as possible*. You should think of these problems as simulated quizzes or exams. *You won't be penalized for mistakes*, so it's in your best interest to do these problems on your own; the detailed feedback we provide will generally be more useful (for you and for us) if you submit your own work.

OCaml Exercises

These problems are from the OCaml Exercises linked on the course webpage.

1. Grouping
- (2) Splitting Rose Trees
- (3) Prefix Map
- (4) Cyclic Function Application
5. Graph Walks

Written Problems

1. Consider the following OCaml program.

```
type 'a foo = Foo of ('a foo -> 'a)
let bar (Foo f) = f
let baz x = bar x x
let out = baz todo
```

Determine the an expression to put in place of `todo` so that `out` evaluates to the integer `42`.

2. Consider the following function.

```
let output (t : task) (x : int) =
  match t with
  | Apply (f, y) -> f x (y + 1)
  | Flip l -> List.map (fun e -> not e) l
  | Filter f -> List.filter (f.pred x) f.elms
  | Reverse l -> List.fold_left (fun b a -> a :: b) [] l
```

Determine the smallest type `task` such that `output` is well-typed.

3. Consider the following pair of functions.

```
let rec foo l =
  match l with
  | [] -> []
  | false :: bs ->
    List.map (fun x -> x - 1) (0 :: foo bs)
  | true :: bs -> bar l
and bar l =
  match l with
  | [] -> []
  | false :: bs -> foo l
  | true :: bs -> List.map ((+) 1) (0 :: bar bs)
```

Determine the type of `foo` and the value of

```
foo [true;true;true;false;false;false;false;true;true] .
```

- (4) Determine a type τ such that the following typing judgment is derivable in 320Caml. Also provide a derivation.

$$\{\} \vdash (\text{fun } x \rightarrow \text{fun } y \rightarrow y \ x) \ (\text{fun } x \rightarrow x) \ (\text{fun } f \rightarrow f \ 1) : \tau$$

- (5) Determine a value v such that $e \Downarrow v$ is derivable in 320Caml. Also provide the derivation.

$$(\text{fun } x \rightarrow x + 1) \ ((\text{fun } x \rightarrow x * x) \ 3)$$